SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE CODE: ITA 06 COURSE NAME: MACHINE LEARNING COURSE

OUTCOME COVERED

CO2: *Neural Network Representation, Perceptron Model, Multilayer Perceptron's, Backpropagation Algorithm, Deep Neural Networks, Genetic Algorithms, Hypothesis Space Search, Genetic Programming, Models of Evaluation and Learning (BL1, BL2)*

Assessment Tool Weightage: 10%

QUESTIONS:3 Total Marks:100 Annexure A

DEBUGGING & OPTIMIZATION TASK QUESTIONS

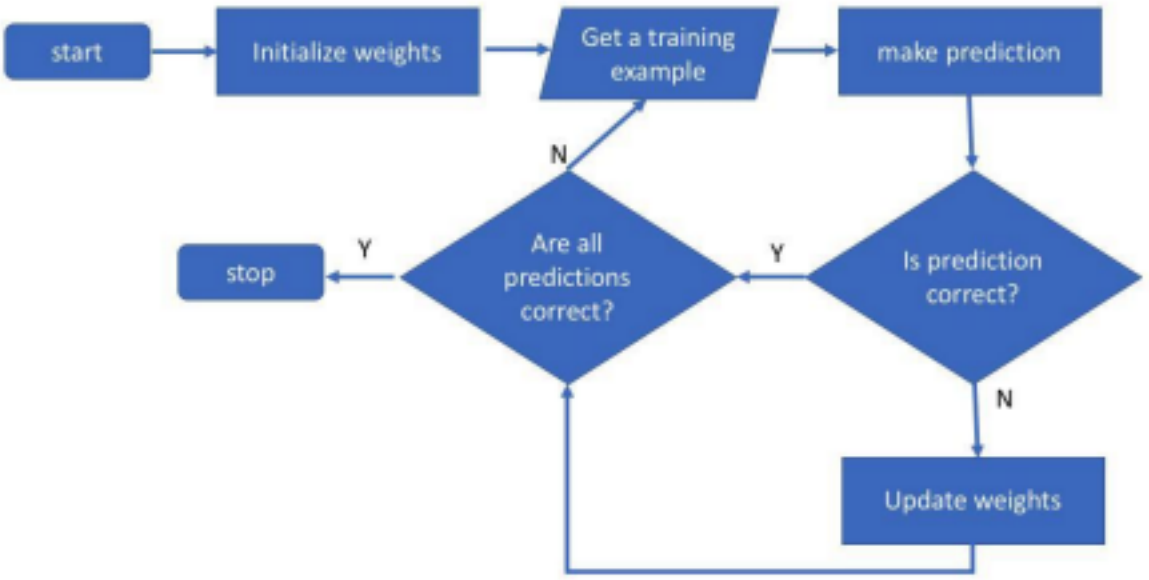
Code of Conduct:

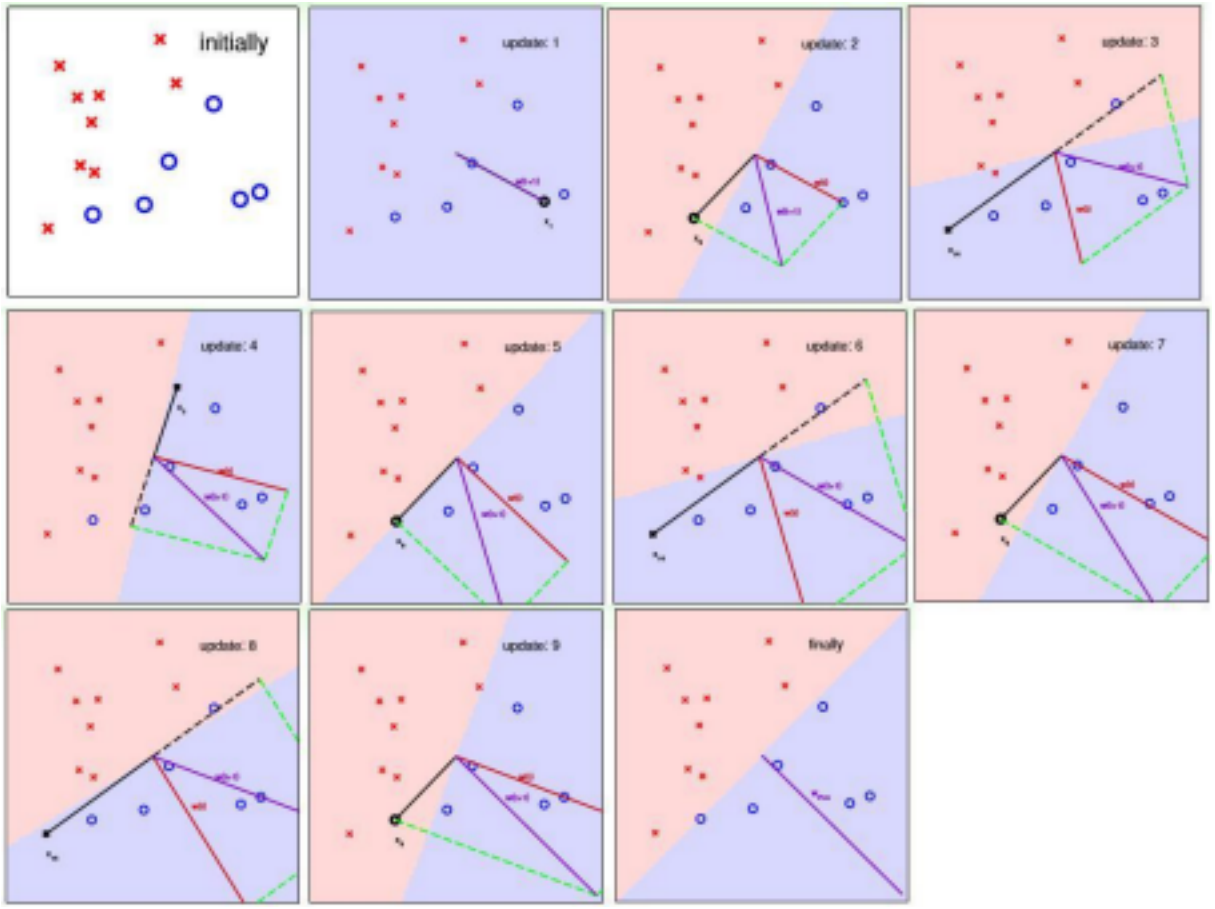
*I **SANTHOSH B/192324146** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

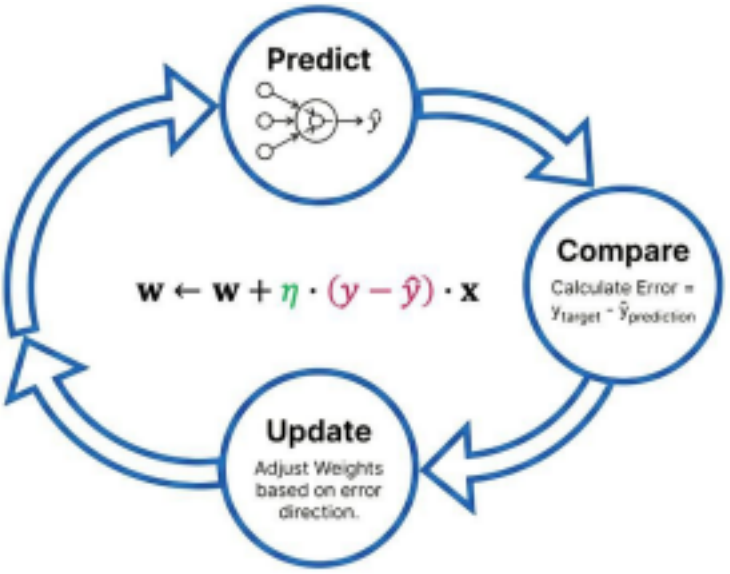
Criteria	*Conceptual Understanding*	*Accuracy of Answers*	*Application of Knowledge*	*Completeness of Response*	*Clarity & Presentation*	*Total*
Weightage	*30%*	*20%*	*30%*	*10%*	*10%*	
Q1						
Q2						
Q3						

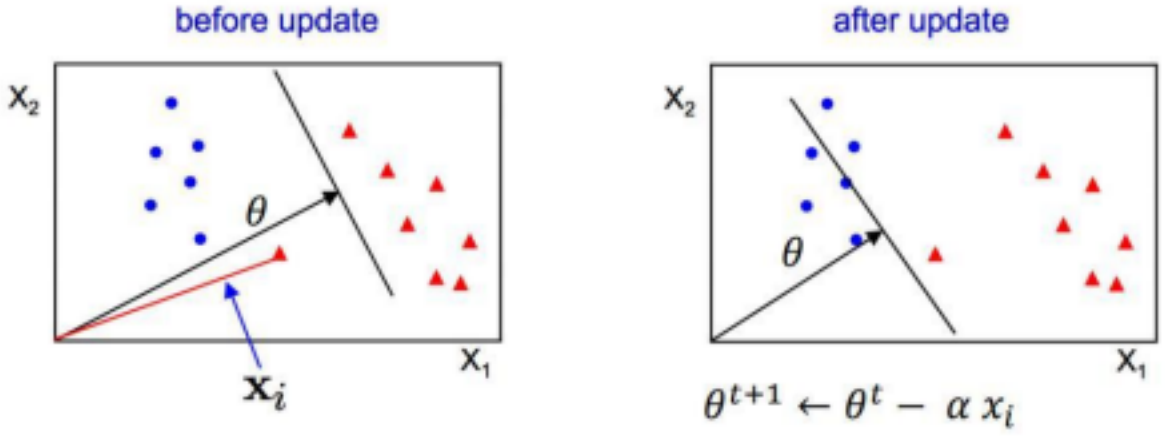
Signature of the student: ***Total Marks Awarded*** *SANTHOSH B*

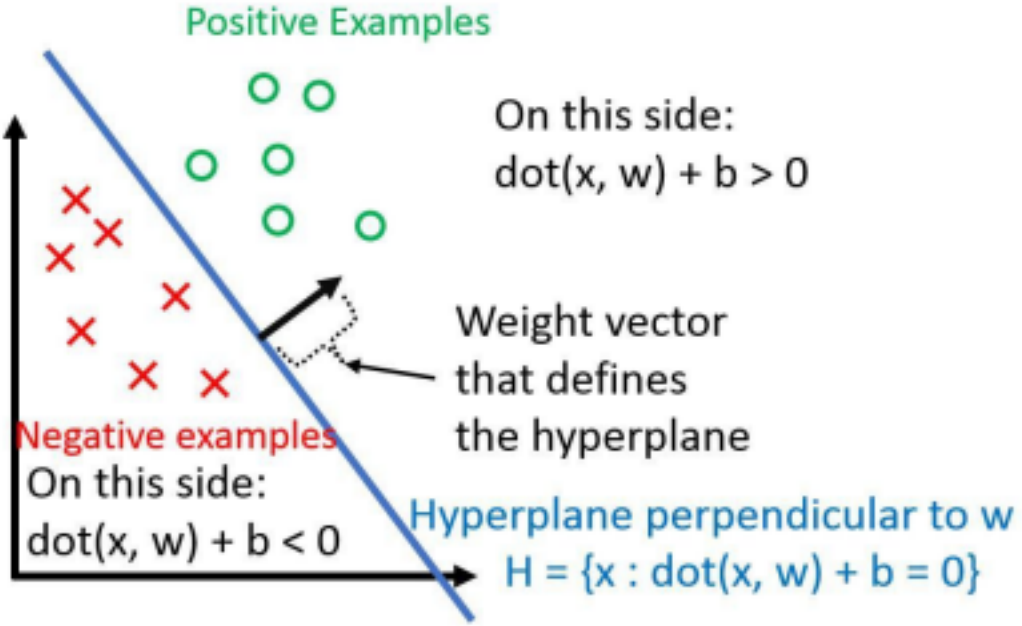
1. A perceptron model is trained on a dataset that is known to be linearly separable, yet the algorithm fails to converge even after many iterations. Analyze the possible causes of this issue, including improper learning rate selection, incorrect weight initialization, bias handling errors, or flawed implementation of the update rule. Propose debugging strategies to identify the root cause and suggest optimization techniques to ensure faster and stable convergence.


start
Initialize weights
Get a training example
make prediction
N
Are all predictions correct?
Y
stop
Y
Is prediction correct?
N
Update weights


initially
update: 1
update: 2
update: 3
update: 4
update: 5
update: 6
update: 7
update: 8
update: 9
finally


Predict
Compare
Calculate Error =
$y_{target} - \hat{y}_{prediction}$
Update
Adjust Weights
based on error
direction.
$\mathbf{w} \leftarrow \mathbf{w} + \eta \cdot (y - \hat{y}) \cdot \mathbf{x}$


before update
after update
X_2
X_1
θ
$\mathbf{x}_i$
$\theta^{t+1} \leftarrow \theta^t - \alpha\, x_i$


Positive Examples
On this side:
dot(x, w) + b > 0
Weight vector
that defines
the hyperplane
Negative examples
On this side:
dot(x, w) + b < 0
Hyperplane perpendicular to w
H = {x : dot(x, w) + b = 0}

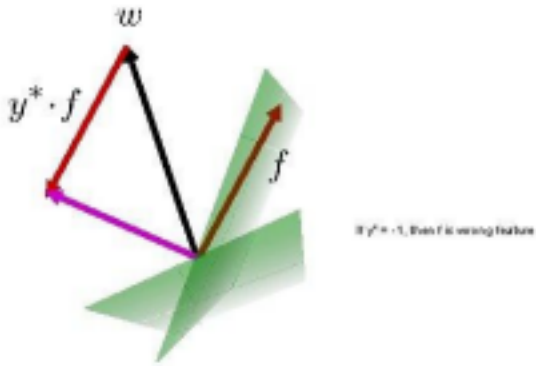
Learning: Binary Perceptron

- Start with weights = 0
- For each training instance:
  - Classify with current weights

$$y = \begin{cases} +1 & \text{if } w \cdot f(x) \geq 0 \\ -1 & \text{if } w \cdot f(x) < 0 \end{cases}$$

  - If correct (i.e., y=y*), no change!
  - If wrong: adjust the weight vector by adding or subtracting the feature vector. Subtract if y* is -1.

$$w = w + y^* \cdot f$$



Possible Causes

1. Improper Learning Rate

- Very high learning rate causes oscillations.
- Very low learning rate results in slow convergence.

2. Incorrect Weight Initialization

- Extremely large or identical initial weights may delay learning.
- Small random weights are preferred.

3. Bias Handling Errors

- Missing or incorrectly updated bias shifts the decision boundary.
- This may prevent correct classification.

4. Incorrect Update Rule

- Wrong implementation of the perceptron update equation:

$$\text{new} = \text{old} + (\quad - \quad)$$

- Incorrect sign or omission of input values leads to failure.

5. Data Issues

- Incorrect labels.
- Data not actually linearly separable.
- Features not properly scaled.

Debugging Strategies

1. Verify that the dataset is truly linearly separable.
2. Check the perceptron update rule implementation.
3. Print weights and bias after each iteration.
4. Plot the decision boundary during training.
5. Monitor classification error after every epoch.
6. Test using a small sample dataset.

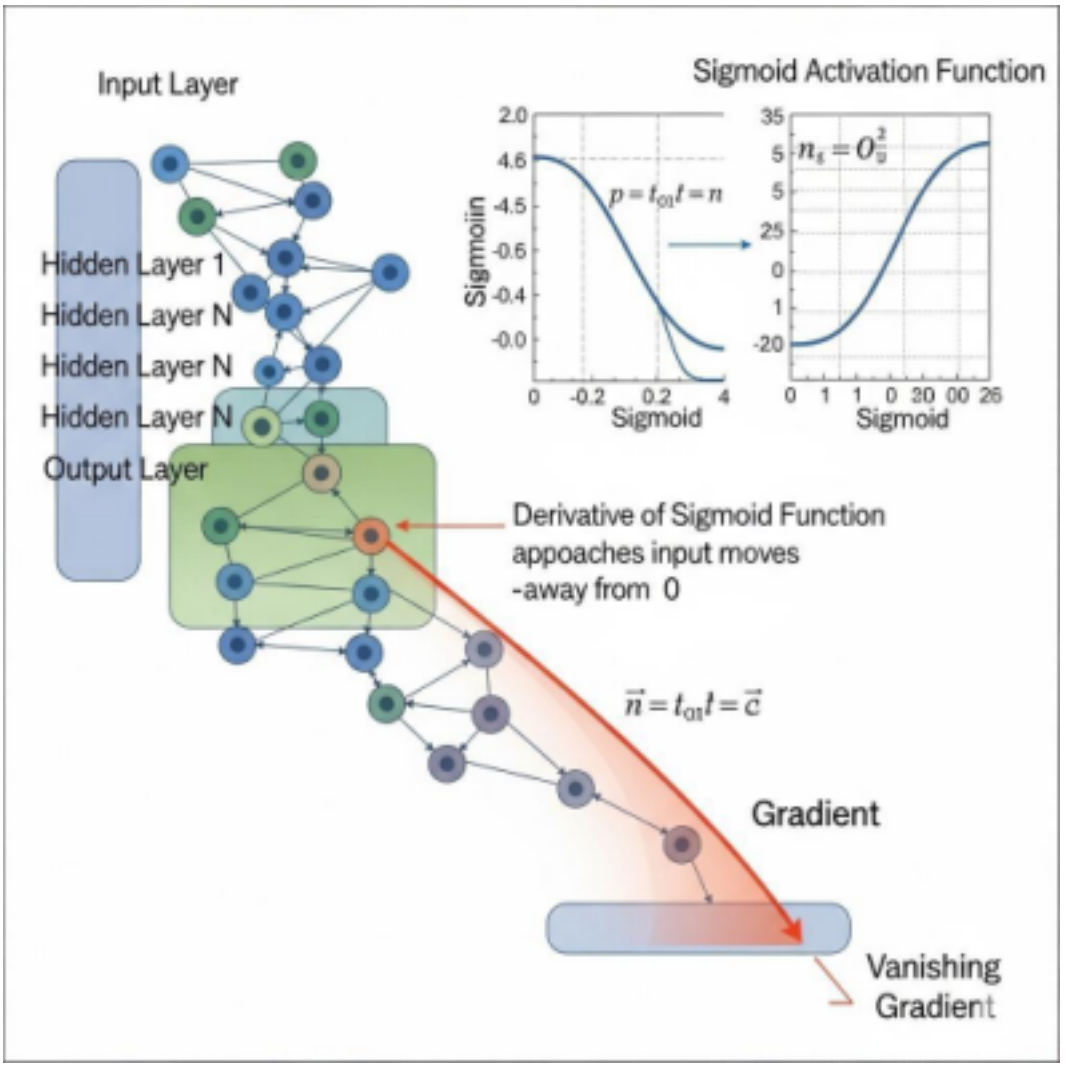
Optimization Techniques

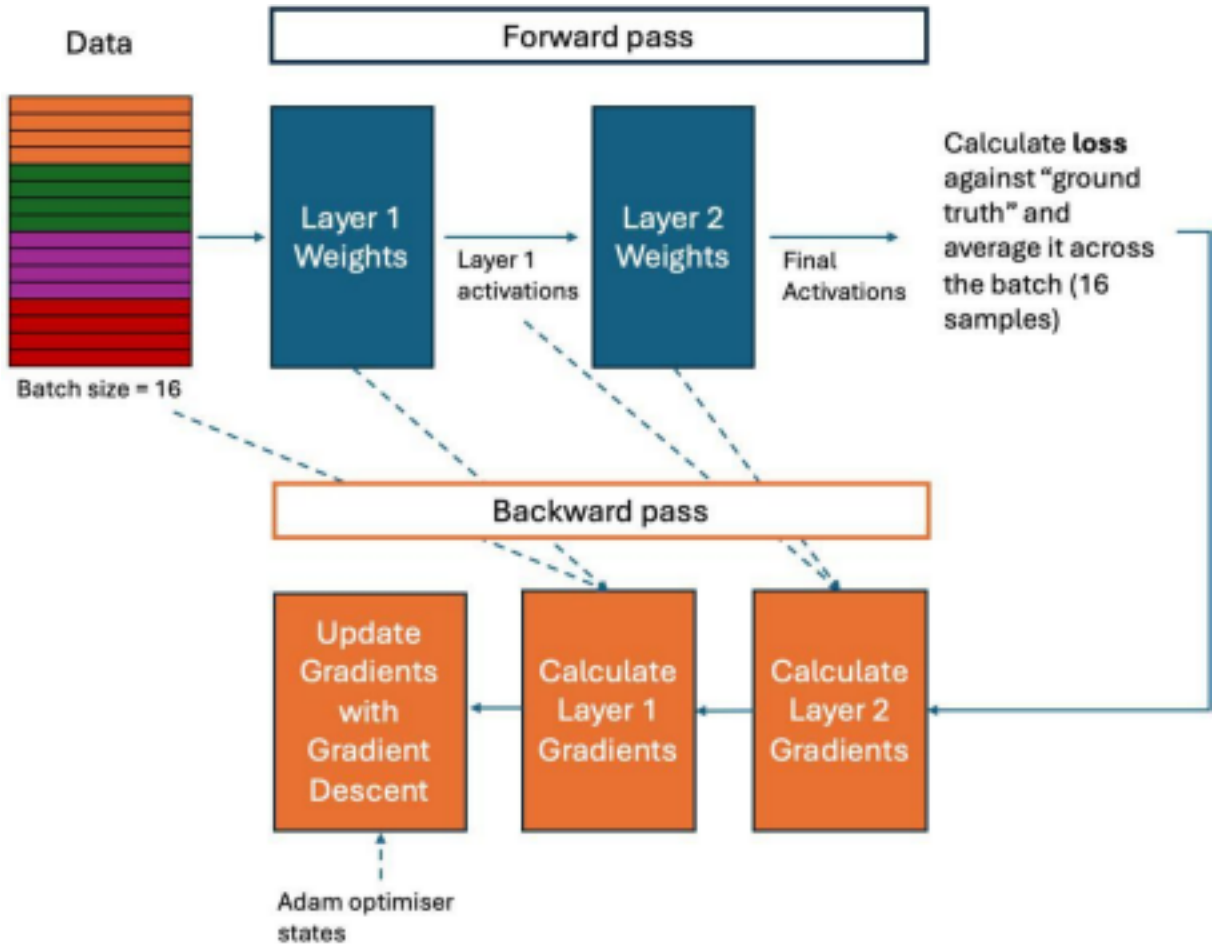
- Normalize input features.
- Use small random weight initialization.
- Select an appropriate learning rate (e.g., 0.01–0.1).
- Shuffle training samples before each epoch.
- Include bias updates correctly.
- Apply early stopping once convergence is achieved.

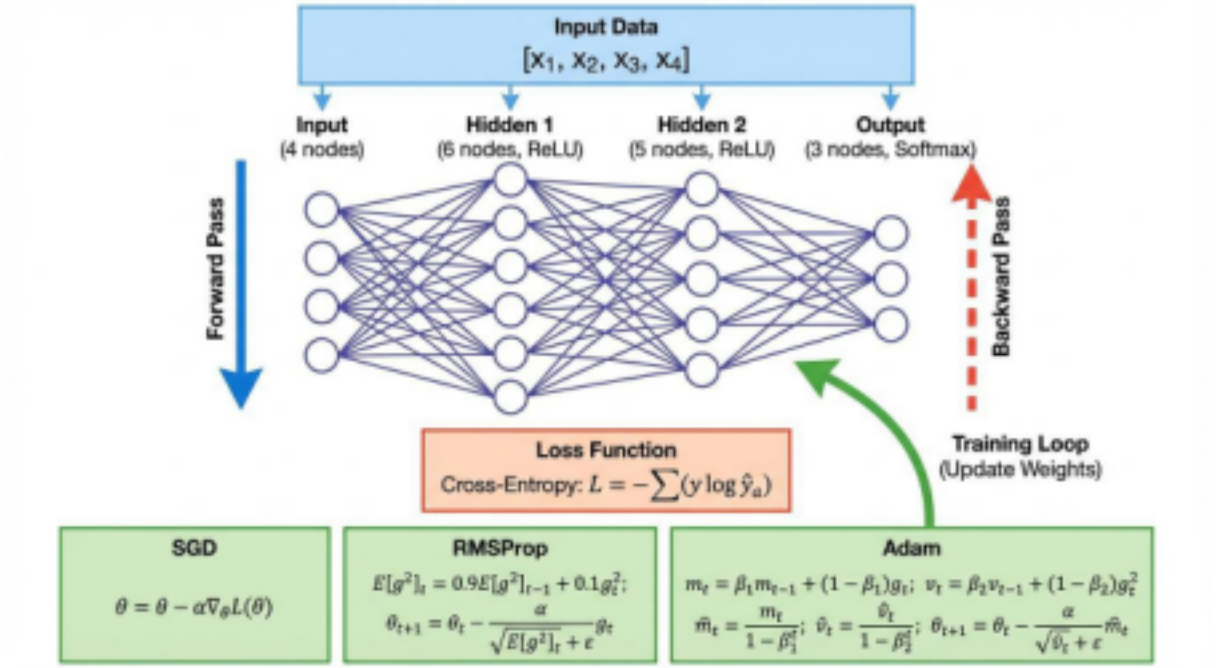
Conclusion

Proper learning rate selection, correct weight updates, bias handling, and normalized input data ensure faster and stable convergence of the perceptron on linearly separable datasets.

2. While training a multilayer perceptron using the backpropagation algorithm, the network exhibits slow convergence and gets stuck in local minima. Examine the potential reasons such as vanishing gradients, inappropriate activation functions, poor weight initialization, or suboptimal learning rates. Describe step-by-step debugging approaches to trace the issue and recommend optimization techniques like adaptive learning rates, normalization, or advanced optimizers to improve performance







Possible Reasons

1. Vanishing Gradients

- **Gradients become extremely small.**
- **Weight updates become negligible.**

2. Inappropriate Activation Functions

- **Sigmoid and tanh may saturate.**
- **ReLU usually provides faster convergence.**

3. Poor Weight Initialization

- **Very large or very small initial weights reduce learning efficiency. 4.**

Incorrect Learning Rate

- **Too high → unstable training.**
- **Too low → very slow convergence.**

5. Overfitting or Underfitting

- **Insufficient or excessive model complexity.**

Step-by-Step Debugging

Step 1: Monitor training and validation loss.
Step 2: Visualize gradients to detect vanishing gradients.
Step 3: Inspect weight distributions after every epoch.
Step 4: Verify activation function outputs.
Step 5: Experiment with different learning rates.
Step 6: Check for data preprocessing issues.

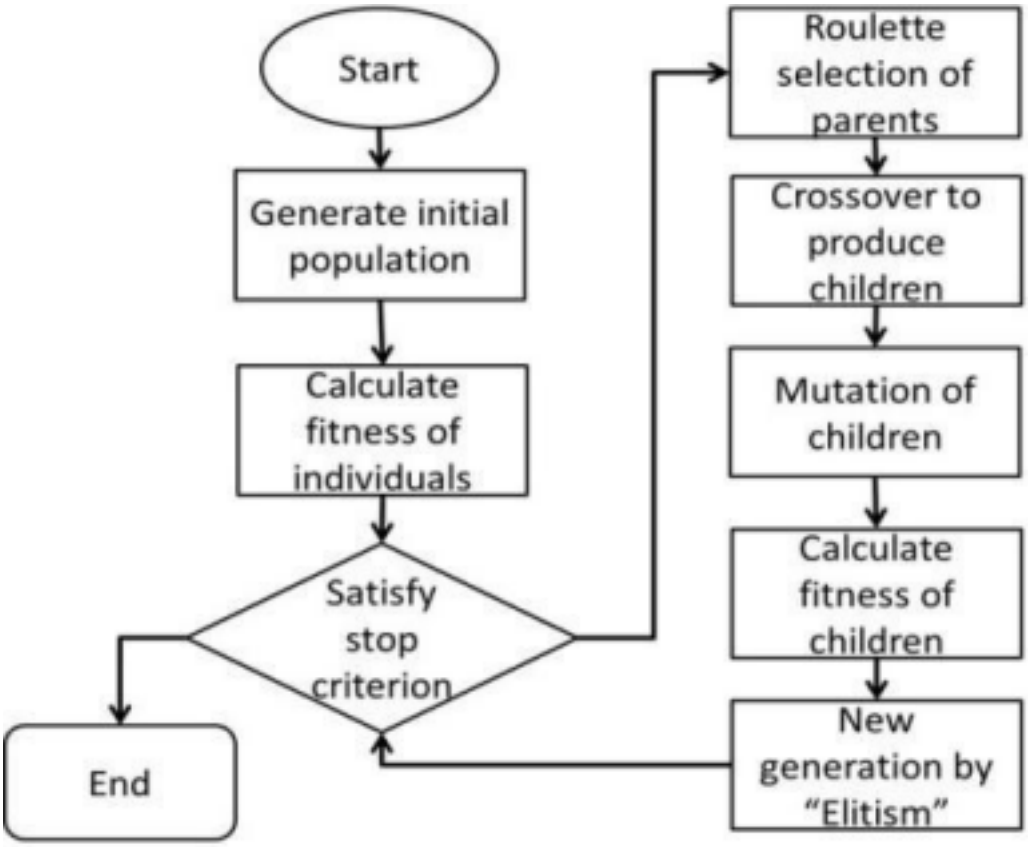
Optimization Techniques

- **Use ReLU or Leaky ReLU activation.**
- **Apply Batch Normalization.**
- **Use Xavier or He Initialization.**
- **Employ adaptive optimizers such as:**
  - **Adam**
  - **RMSProp**
  - **AdaGrad**
- **Implement learning rate scheduling.**
- **Use mini-batch gradient descent.**
- **Apply dropout to reduce overfitting.**

Conclusion

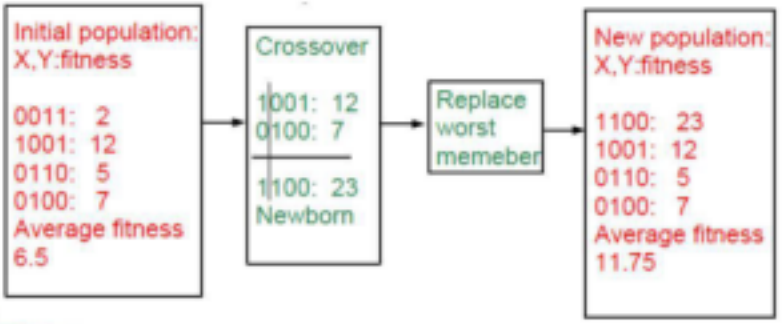
Proper activation functions, suitable initialization, adaptive learning rates, and normalization significantly improve convergence speed and help the network avoid local minima.

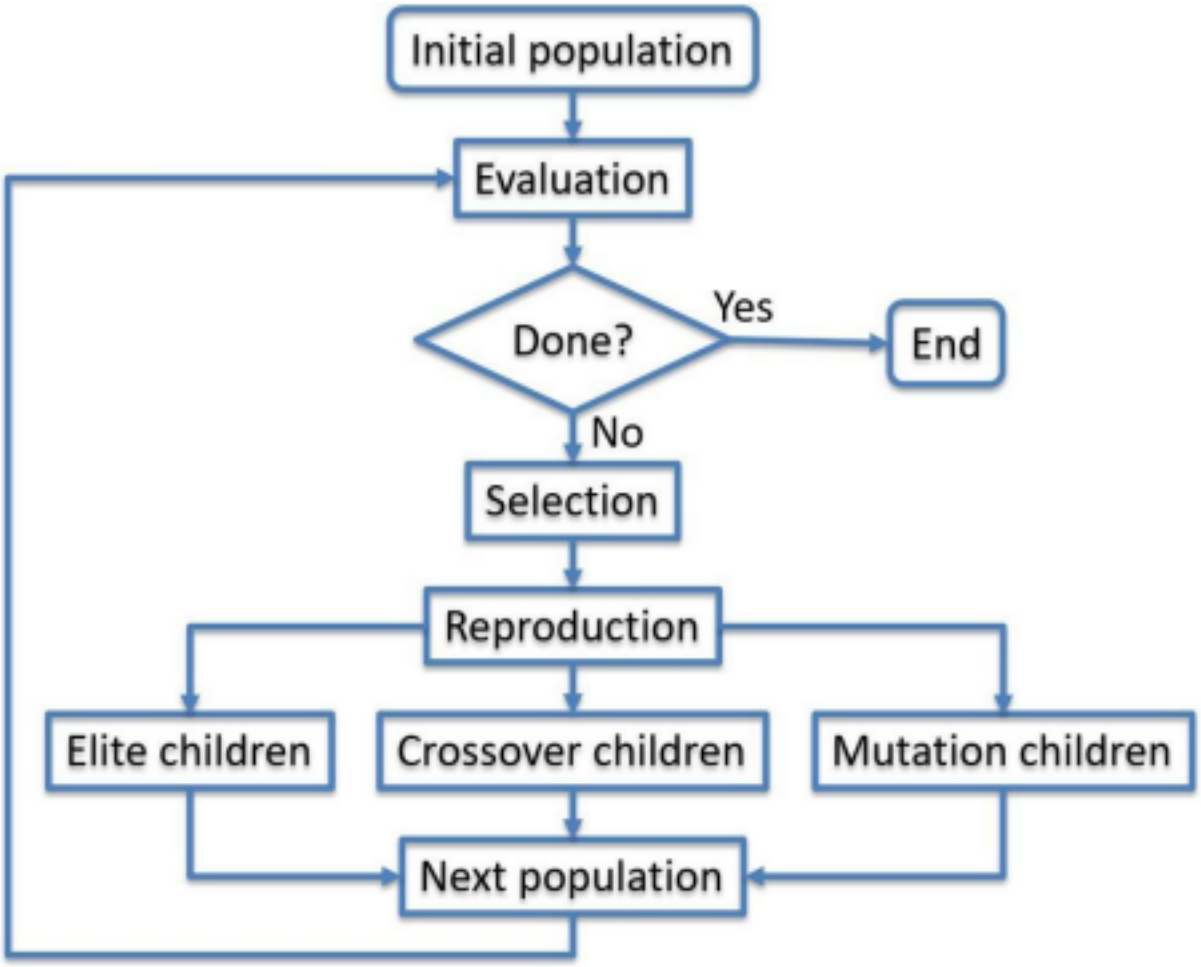
3. A genetic algorithm designed for hypothesis space search is producing inconsistent and suboptimal solutions across multiple runs. Investigate possible issues such as improper encoding of chromosomes, incorrect fitness function design, premature convergence, or lack of diversity in the population. Explain how to debug these problems systematically and suggest optimization strategies like mutation tuning, selection pressure adjustment, and elitism to enhance solution quality and convergence speed.



Example with binary representation

- **maximize 2X^2-y+5 where X:[0,3],Y:[0,3]**





Possible Issues

1. Improper Chromosome Encoding
- Incorrect representation of solutions.
- Genetic operators become ineffective.

2. Incorrect Fitness Function
- Does not accurately measure solution quality.
- Leads the search toward poor solutions.

3. Premature Convergence
- Population quickly becomes similar.
- Exploration stops too early.

4. Low Population Diversity
- Limited variation reduces search capability.

5. Poor Parameter Settings
- Mutation rate too low or too high.
- Excessive selection pressure.

Debugging Process

Step 1: Verify chromosome representation.
Step 2: Validate the fitness function using known solutions.
Step 3: Track best and average fitness across generations.
Step 4: Monitor population diversity.
Step 5: Test crossover and mutation independently.
Step 6: Repeat experiments with different random seeds.

Optimization Strategies
- Increase mutation rate moderately.
- Use tournament or roulette-wheel selection.
- Apply elitism to preserve the best solutions.

- Increase population size.
- Improve chromosome encoding.
- Use adaptive mutation and crossover probabilities.
- Introduce random individuals periodically to maintain diversity.

Conclusion

A well-designed chromosome representation, accurate fitness function, balanced mutation and selection parameters, and elitism improve solution quality, maintain diversity, and accelerate convergence of genetic algorithms.